SCHOOL OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

Devi Ahilya Vishwavidyalaya (DAVV), Indore

Programme: M.Tech (Information Architecture & Software Engineering)

LABORATORY MANUAL

Information System Design

Course Code: CS-4407

Semester 1, Year 1

Submitted By:

Name: ROHIT BHURE

Roll No.: 2610611

Programme: M.Tech (IASE)

Semester: SEM 1 (Year 1)

Submitted To:

Name: Dr. Archana Thakur

Department: School of Computer Science & Information Technology

_____

Signature

Academic Session: 2026–2027

CERTIFICATE

This is to certify that **ROHIT BHURE**, Roll No. **2610611**, has satisfactorily completed all the experiments of Information System Design (Course Code: **CS-4407**) as prescribed by Devi Ahilya Vishwavidyalaya (DAVV) for M.Tech (Information Architecture & Software Engineering), Semester 1, during the academic session 2026–2027.

_____ _____

Faculty / Lab Incharge
Name: Dr. Archana Thakur
Date: _____

INDEX

Exp. No.	Title of Experiment	Page No.	Date	Signature
1	Identify the different components in the following systems: Library Information System, Microsoft Office and Cellular Phone.	3		
2	Identify systems that are not information systems.	5		
3	Use the structured design methodology to produce a design for an ASCII to EBCDIC converter, a basic arithmetic calculator, a student registration system and a hardware store inventory system.	6		
4	Design an information system for a college to manage student records.	9		
5	Design an information system for a library.	13		
6	Create a Use Case Diagram of the Library Management System using a UML tool.	18		
7	Draw a Class Diagram using a UML tool.	20		
8	Create a Sequence Diagram for the Book Issue process using a UML tool.	22		
9	Create a Collaboration (Communication) Diagram using a UML tool.	24		
10	Draw an Activity Diagram for an ATM withdrawal process using a UML tool.	25		
11	Draw a State Chart Diagram for modeling the states of an ATM card using a UML tool.	27		
12	Create a Component Diagram using a UML tool.	28		
13	Create a Deployment Diagram using a UML tool.	30		
14	Import Java code into Rational Rose and generate the class diagram.	31		
15	Create a class diagram and generate Java source code.	33		
16	Create a new Rational Rose project, save the model and add a new diagram.	35		

Experiment No. 1

Identify the different components in the following systems: Library Information System, Microsoft Office and Cellular Phone.

Aim:

To identify the components (hardware, software, data, people, procedures, inputs, processes and outputs) of a Library Information System, Microsoft Office and a Cellular Phone.

Software / Hardware Required:

Reference study of the given systems; MS Word / text editor for documentation; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A system is a set of interrelated components that work together to achieve a common goal. An information system collects (input), processes, stores and distributes (output) data as information to support the operations and decisions of an organisation, and it uses feedback to control itself. The main components of an information system are hardware, software, data, people and procedures. Every system also has a boundary and an environment, and it can be studied by identifying its inputs, processes and outputs.

Procedure / Algorithm:

1. Select the three systems: Library Information System, Microsoft Office and Cellular Phone.
2. For each system identify the hardware, software, data, people and procedures.
3. Identify the inputs, processes and outputs of each system.
4. Present the components in a comparison table.

Components of the Systems:

Component	Library Information System	Microsoft Office	Cellular Phone
Hardware	Server, library PCs, barcode scanner, printer, RFID gates	User's PC / laptop, keyboard, mouse, monitor, printer	Handset, processor, battery, touch screen, antenna, SIM card, camera
Software	Library management software, database (MySQL), OS	Word, Excel, PowerPoint, Outlook, Access and the operating system	Mobile OS (Android / iOS), dialer, messaging and other apps, firmware
Data	Books, members, issue / return records, fines	Documents, worksheets, presentations, e-mails, templates	Contacts, call log, messages, photos, settings, network data
People	Librarian, library staff, members, administrator	Students, office staff, managers, other end users	Subscriber, network operator staff, service providers
Procedures	Membership rules, issue / return rules, fine rules, backup	Create / edit / save / share documents, licensing and updates	Making a call, sending SMS, recharge, SIM and security procedures
Inputs	Member details, book details, issue and return requests	Typed text, numbers, images, commands from keyboard or mouse	Dialled number, touch input, voice, SMS text, incoming signals
Processes	Search, issue, return, fine calculation, report generation	Formatting, calculation, formulas, charts, spell check, saving	Signal processing, call set-up, encoding / decoding, data transfer
Outputs	Issue slip, fine receipt, overdue list, reports	Printed or on-screen documents, charts, slides, e-mails	Voice, display, ringtone, messages, notifications

Output:

All three systems have the five basic components (hardware, software, data, people and procedures) and work on the input - process - output model. The Library Information System and Microsoft Office are information systems for managing information, while the Cellular Phone is a communication system which also contains information-system functions (contacts, messages).

Result:

The components of the Library Information System, Microsoft Office and the Cellular Phone were identified and tabulated.

Conclusion:

Identifying hardware, software, data, people and procedures gives a clear structure of any system and is the first step of system design.

Experiment No. 2

Identify systems that are not information systems.

Aim:

To identify systems that are not information systems and to give the reason by comparing them with information systems.

Software / Hardware Required:

Reference study of the given systems; MS Word / text editor for documentation; PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A system is an information system only if it collects, processes, stores and distributes data as information for the use of people or organisations in operations and decision making. Many systems exist that transform energy or matter, or work by natural or mechanical laws, but do not handle information as their purpose; these are physical, natural or mechanical systems and not information systems. Examples are the solar system, the human digestive system, a bicycle or a ceiling fan.

Procedure / Algorithm:

1. Write the definition and the test of an information system (input, processing, storage, output of information for decisions).
2. List common systems: natural, mechanical and information based.
3. Apply the test to each system.
4. Classify each system with a reason and give a few examples of real information systems.

Classification of Systems:

System	Information system?	Reason
Solar system	No	Natural system of planets; it does not collect or process information for users.
Human digestive system	No	Biological system that converts food into energy; no information is processed for decisions.
Bicycle	No	Mechanical system that transfers muscle power into motion; it has no data input or output.
Ceiling fan	No	Electro-mechanical system that rotates blades; it does not store or process information.
Water cycle / river system	No	Natural physical process; it produces no information for human decisions.
Mechanical wall clock	No	It only shows time by gears; there is no data input, storage or processing (a digital time-keeping app is different).
Library management system	Yes	Stores books and members, processes issue / return and produces reports.
ATM / banking system	Yes	Takes requests, processes transactions, stores balances and gives receipts.
Online reservation system	Yes	Collects passenger data, processes bookings and produces tickets.

Output:

Systems 1 to 6 (solar system, digestive system, bicycle, ceiling fan, water cycle and mechanical clock) are not information systems. Systems 7 to 9 satisfy the input - processing - storage - output of information test and are information systems.

Result:

The systems that are not information systems were identified and the reason for each was given.

Conclusion:

A system is an information system only when its purpose is to collect, process, store and supply information; other systems are physical, natural or mechanical.

Experiment No. 3

Use the structured design methodology to produce a design for an ASCII to EBCDIC converter, a basic arithmetic calculator, a student registration system and a hardware store inventory system.

Aim:

To apply the structured design methodology and produce a modular design (structure chart) for four systems: an ASCII to EBCDIC converter, a basic arithmetic calculator, a student registration system and a hardware store inventory system.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

Structured design is a top-down methodology in which a system is decomposed into small, independent modules arranged in a hierarchy. The result is shown as a structure chart in which each box is a module, lines show the calling relationship and the top module controls the others. Good modules have high cohesion (a single function) and low coupling (little dependence on other modules). The usual steps are: define the problem, identify the inputs and outputs, draw the data flow, divide the system into input, transform and output modules, and refine each module.

Procedure / Algorithm:

1. Define the problem, inputs and outputs of each of the four systems.
2. Identify the main functions: input, processing (transform) and output.
3. Divide each main function into lower-level modules (top-down decomposition).
4. Draw the structure chart of each system, with the controlling module at the top.
5. Describe each module with its function and check cohesion and coupling.

Module Design:

(a) ASCII to EBCDIC converter - input: ASCII character; output: EBCDIC code

Module	Function
Convert ASCII to EBCDIC	Controlling module; calls the three modules below in sequence
Read ASCII Character	Reads one character from the input
Lookup EBCDIC Code	Finds the EBCDIC code from a conversion table; uses Validate ASCII Range
Validate ASCII Range	Checks that the character code is between 0 and 127
Write EBCDIC Character	Displays or stores the converted code

(b) Calculator with basic arithmetic functions - input: two operands and operator; output: result

Module	Function
Calculator	Controlling module
Read Input	Calls Read Operands and Read Operator
Perform Operation	Selects Add, Subtract, Multiply or Divide according to the operator
Divide	Divides the operands after Check Divide by Zero
Display Result	Shows the result or an error message

(c) Student registration - input: student and course details; output: registration receipt

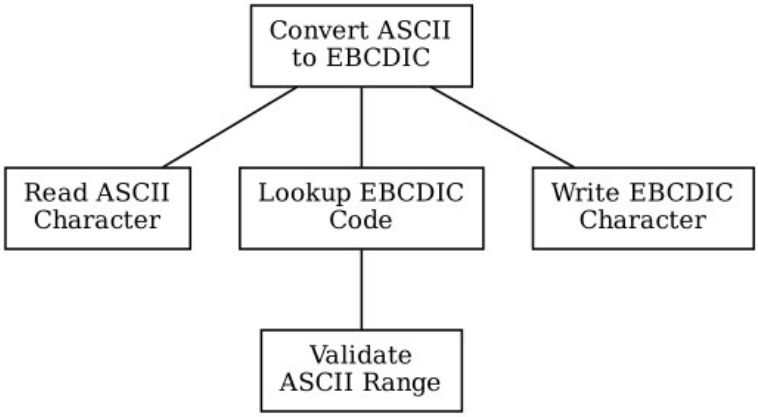
Module	Function
Student Registration	Controlling module
Get Student Details	Accepts name, roll number and programme
Verify Eligibility	Checks admission status; calls Check Fee Payment
Select Courses	Lists courses and calls Check Seat Availability
Save Registration	Stores the registration record
Print Receipt	Prints the registration slip

(d) Inventory of a hardware store - input: item and stock transactions; output: stock and reorder reports

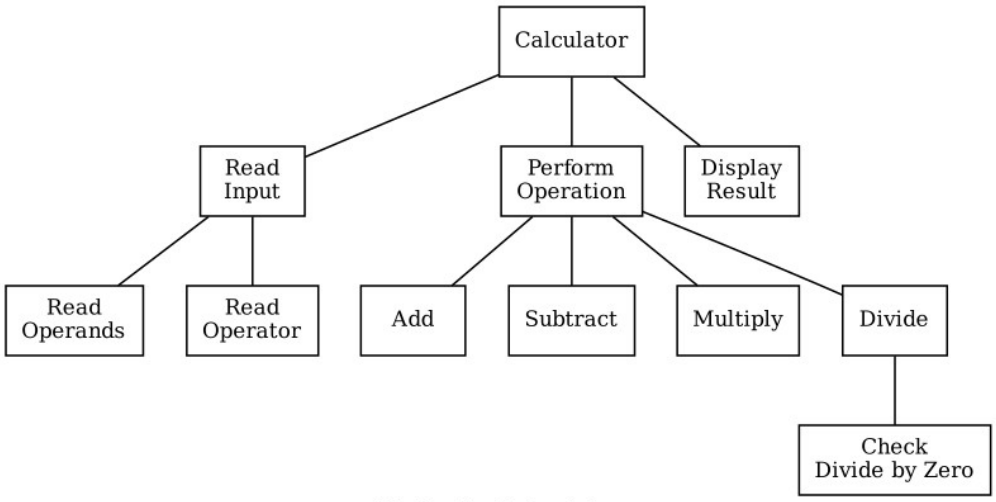
Module	Function
Inventory Manager	Controlling module
Maintain Items	Adds, updates and deletes item records
Stock Transactions	Receive Stock increases and Sell / Issue Stock decreases the quantity
Reorder Check	Compares quantity with the reorder level
Reports	Produces the stock report and the low-stock report

Output:

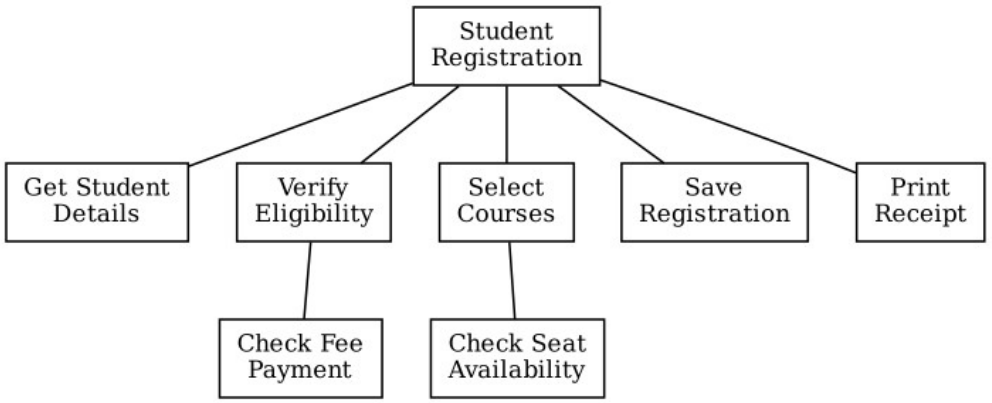
The structure charts of the four systems are shown below. Each system has one controlling module and separate input, processing and output modules.



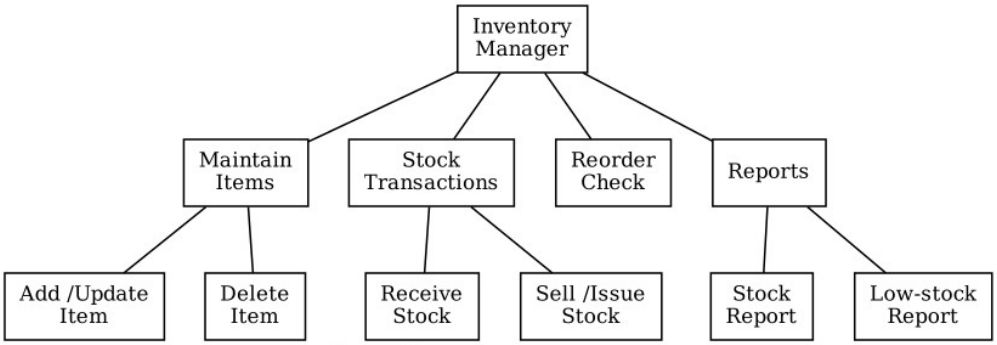
(a) ASCII to EBCDIC Converter



(b) Basic Calculator



(c) Student Registration



(d) Hardware Store Inventory

Result:

The structured design of the four systems was prepared and the structure charts were drawn.

Conclusion:

Top-down decomposition into cohesive, loosely coupled modules makes a system easier to understand, build and maintain.

Experiment No. 4

Design an information system for a college to manage student records.

Aim:

To design an information system for a college to manage student records by identifying the inputs, processes and outputs, drawing the context diagram, DFD level 0 and ER diagram, and creating the database tables with primary and foreign keys.

Software / Hardware Required:

UML / diagram tool (StarUML, Rational Rose or draw.io), MS Word; MySQL 8.0 and MySQL Workbench for the tables; PC with minimum 4 GB RAM.

Theory:

System design begins with the inputs, processes and outputs. A context diagram shows the whole system as a single process with the external entities and the data flows between them. A Data Flow Diagram (DFD) level 0 breaks the system into its main processes, data stores and flows. An Entity Relationship (ER) diagram models the data as entities, attributes and relationships, and is converted into relational tables in which the primary key identifies a row and a foreign key links a table to its parent table.

Procedure / Algorithm:

1. Identify the inputs, processes and outputs of the college system.
2. Draw the context diagram with the external entities Student, Faculty and Administrator.
3. Draw DFD level 0 with processes, data stores and data flows.
4. Design the ER diagram with entities, attributes, keys, relationships and cardinalities.
5. Convert the ER diagram into tables with primary and foreign keys and create them in MySQL.

Design Details and Database Tables:

Inputs, processes and outputs

Inputs	Processes	Outputs
Student admission form and personal details	Register student and generate ID	Student ID card, admission receipt
Course details and enrolment requests	Maintain (add / update / delete) student records	Updated student records
Attendance and marks from faculty	Record attendance and marks	Attendance sheet, marksheet
Report requests	Generate reports	Student list, result and statistical reports

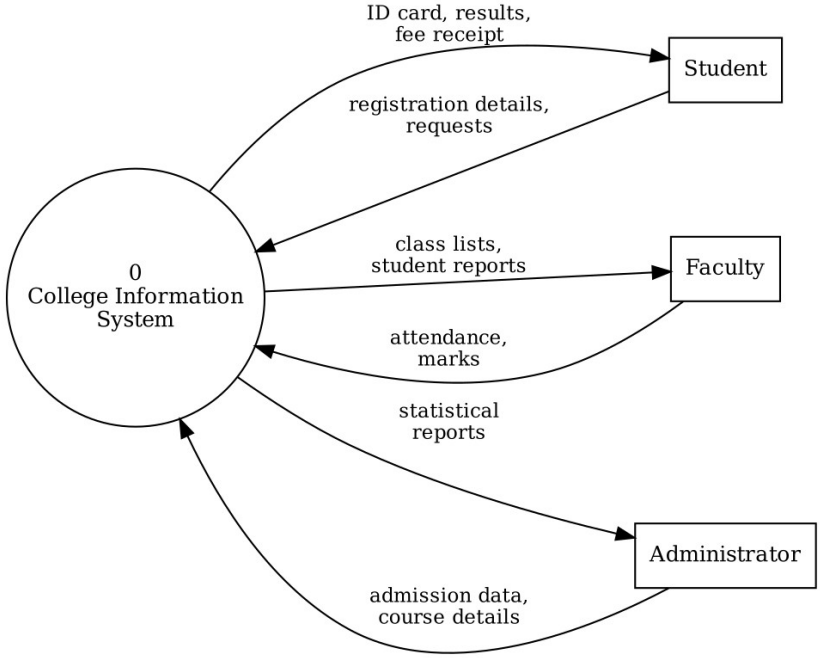
Database tables (MySQL)

```
CREATE TABLE department (
  dept_id   INT PRIMARY KEY,
  dept_name VARCHAR(40) NOT NULL UNIQUE
);
CREATE TABLE student (
  student_id INT PRIMARY KEY,
  name       VARCHAR(50) NOT NULL,
  email      VARCHAR(60) UNIQUE,
  phone      VARCHAR(10),
  dob        DATE,
  dept_id    INT NOT NULL,
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
CREATE TABLE faculty (
  faculty_id  INT PRIMARY KEY,
  name        VARCHAR(50) NOT NULL,
  designation VARCHAR(30),
  dept_id     INT NOT NULL,
  FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

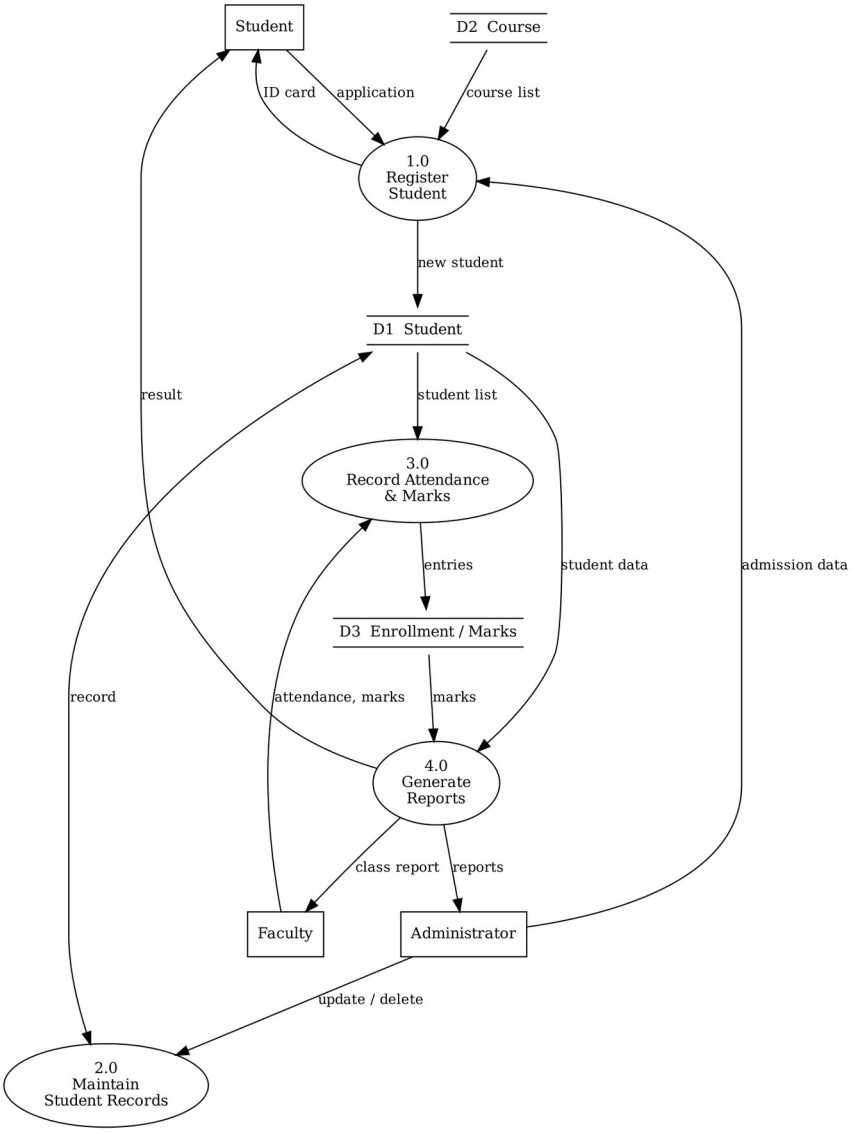
```
CREATE TABLE course (
  course_id  VARCHAR(10) PRIMARY KEY,
  title      VARCHAR(60) NOT NULL,
  credits    INT CHECK (credits > 0),
  dept_id    INT NOT NULL,
  faculty_id INT,
  FOREIGN KEY (dept_id)    REFERENCES department(dept_id),
  FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id)
);
CREATE TABLE enrollment (
  student_id  INT,
  course_id   VARCHAR(10),
  enroll_date DATE,
  grade       CHAR(2),
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES student(student_id),
  FOREIGN KEY (course_id)  REFERENCES course(course_id)
);
```

Output:

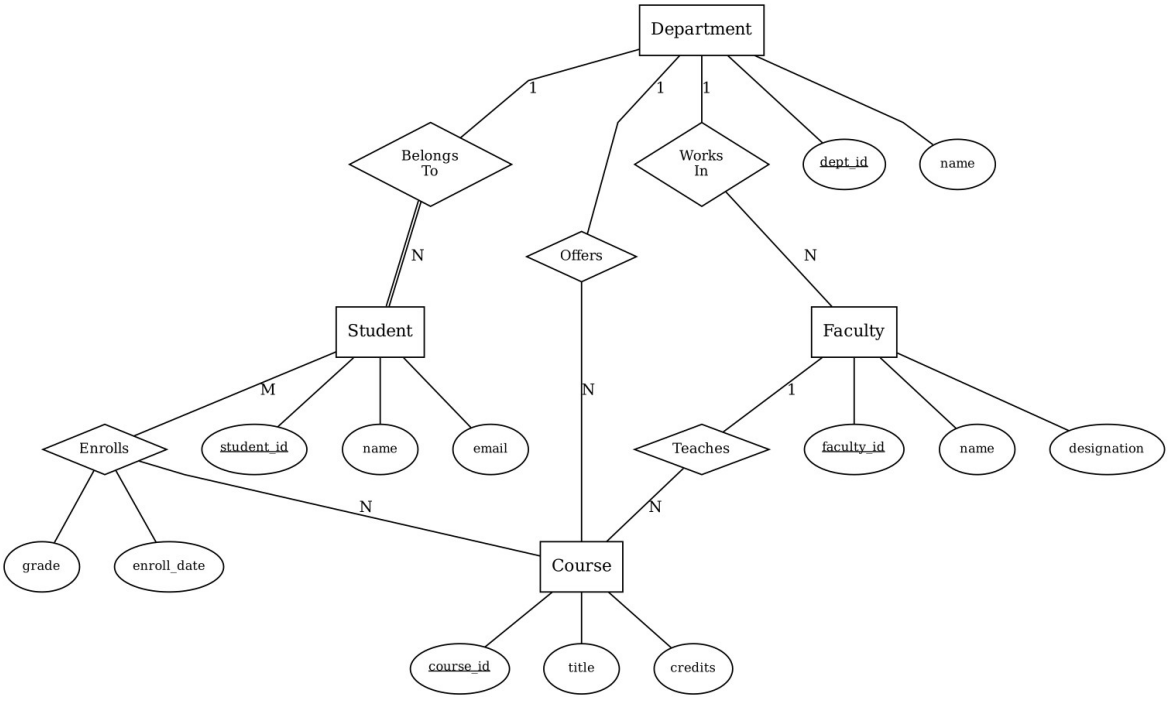
The context diagram, DFD level 0 and ER diagram of the College Information System are given below. The five tables were created in MySQL with primary keys (student_id, course_id, dept_id, faculty_id, and the composite key student_id + course_id of enrollment) and foreign keys.



Context Diagram



DFD Level 0



ER Diagram

Result:

The college student records system was designed with the context diagram, DFD level 0, ER diagram and database tables.

Conclusion:

A systematic design from inputs and outputs to DFD, ER diagram and tables gives a correct database design for the college.

Experiment No. 5

Design an information system for a library.

Aim:

To design an information system for a library by identifying the system requirements, drawing the context diagram, DFD level 0 and level 1 and the ER diagram, and designing the tables for Books, Members, Borrowing and Staff.

Software / Hardware Required:

UML / diagram tool (StarUML, Rational Rose or draw.io), MS Word; MySQL 8.0 and MySQL Workbench for the tables; PC with minimum 4 GB RAM.

Theory:

A library information system keeps the records of books, members and staff and controls the issue and return of books. The requirements are first collected, then the context diagram shows the system and its external entities, DFD level 0 shows the main processes and DFD level 1 explains one process (here Issue and Return) in more detail. The ER diagram shows the entities and relationships, and from it the tables Books, Members, Staff and Borrowing are derived; Borrowing is the table of the M:N relationship between Members and Books and contains foreign keys to Members, Books and Staff.

Procedure / Algorithm:

1. Collect the system requirements (functional and non-functional).
2. Draw the context diagram with the entities Member, Librarian and Book Supplier.
3. Draw DFD level 0 with four main processes and DFD level 1 for the process Issue and Return.
4. Draw the ER diagram of Member, Book, Staff, Author and the relationship Borrows.
5. Design and create the tables Books, Members, Staff and Borrowing with primary and foreign keys.

Requirements and Tables:

System requirements

ID	Requirement
R1	Maintain the details of books, authors and available copies.
R2	Register members and maintain membership records.
R3	Issue a book to a member only if the member is valid and a copy is available.
R4	Record the return of a book and calculate the fine for late return.
R5	Maintain the staff details and record which staff member handled a transaction.
R6	Generate reports: available books, overdue books, member history.
R7	Non-functional: secure login, data integrity, quick search (under 2 seconds).

Table design

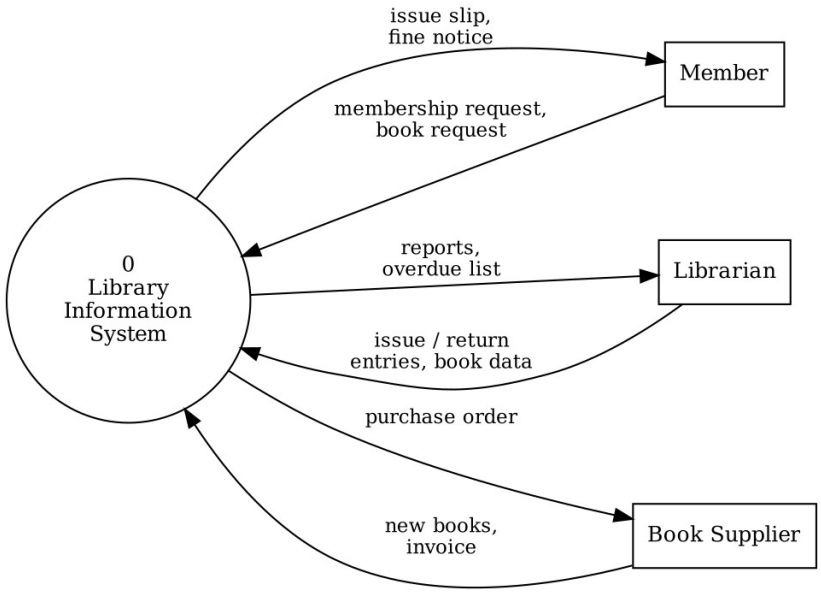
Table	Columns	Keys
books	book_id, title, author, isbn, category, copies_available	PK: book_id; isbn unique
members	member_id, name, phone, email, join_date	PK: member_id; email unique
staff	staff_id, name, role	PK: staff_id
borrowing	borrow_id, member_id, book_id, staff_id, issue_date, due_date, return_date, fine	PK: borrow_id; FK: member_id, book_id, staff_id

```
CREATE TABLE books (
  book_id          INT PRIMARY KEY,
  title            VARCHAR(80) NOT NULL,
  author           VARCHAR(50),
  isbn             VARCHAR(15) UNIQUE,
  category         VARCHAR(30),
  copies_available INT CHECK (copies_available >= 0)
);
CREATE TABLE members (
  member_id INT PRIMARY KEY,
  name      VARCHAR(50) NOT NULL,
```

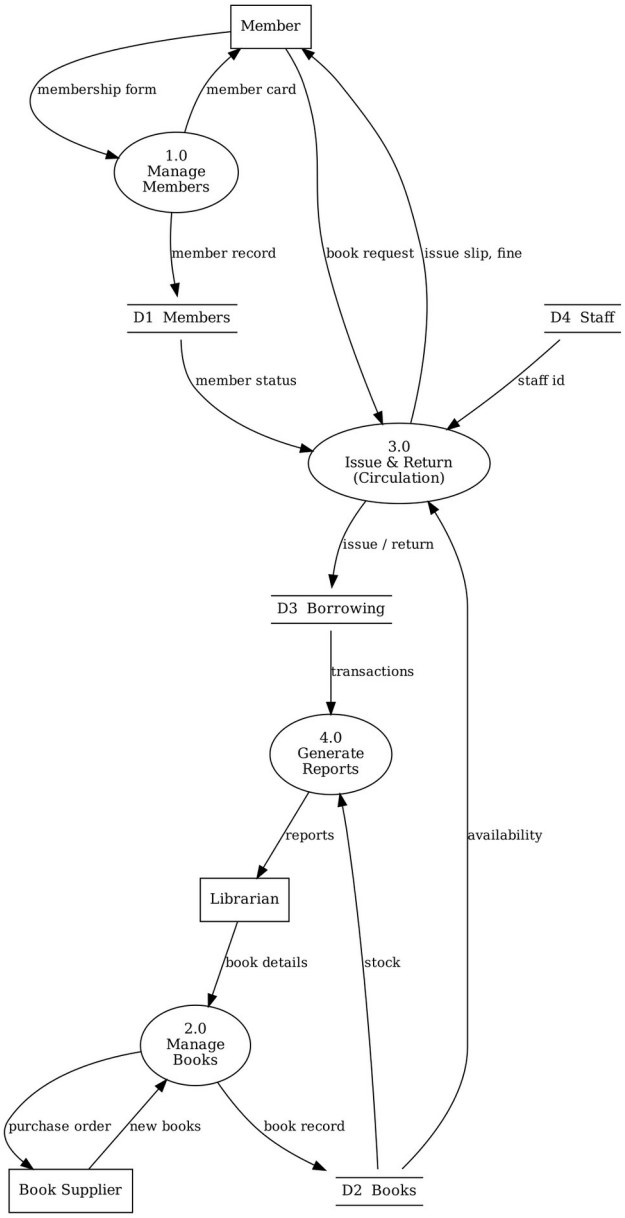
```
  phone     VARCHAR(10),
  email     VARCHAR(60) UNIQUE,
  join_date DATE
);
CREATE TABLE staff (
  staff_id INT PRIMARY KEY,
  name     VARCHAR(50) NOT NULL,
  role     VARCHAR(30)
);
CREATE TABLE borrowing (
  borrow_id   INT PRIMARY KEY,
  member_id   INT NOT NULL,
  book_id     INT NOT NULL,
  staff_id    INT NOT NULL,
  issue_date  DATE NOT NULL,
  due_date    DATE NOT NULL,
  return_date DATE,
  fine        DECIMAL(6,2) DEFAULT 0,
  FOREIGN KEY (member_id) REFERENCES members(member_id),
  FOREIGN KEY (book_id)   REFERENCES books(book_id),
  FOREIGN KEY (staff_id)  REFERENCES staff(staff_id)
);
```

Output:

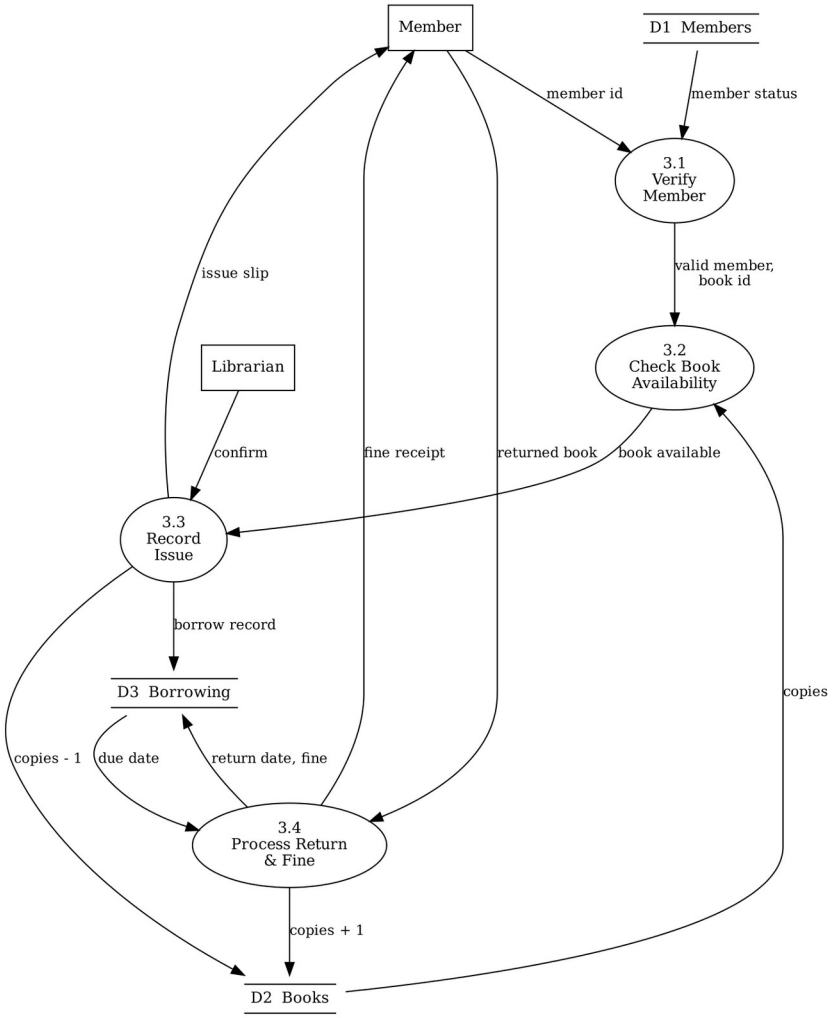
The context diagram, DFD level 0, DFD level 1 and ER diagram of the Library Information System are given below. The four tables were created in MySQL; borrowing has three foreign keys.



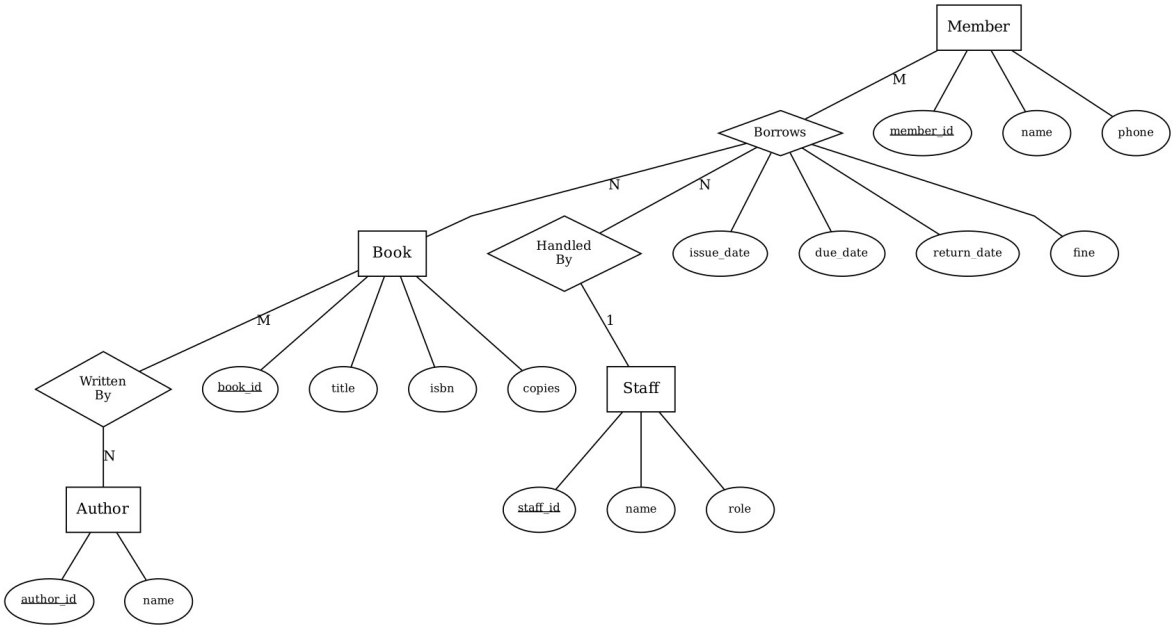
Context Diagram


Member
membership form
member card
1.0
Manage
Members
member record
book request
issue slip, fine
D1 Members
D4 Staff
member status
staff id
3.0
Issue & Return
(Circulation)
issue / return
D3 Borrowing
transactions
4.0
Generate
Reports
reports
availability
Librarian
book details
stock
2.0
Manage
Books
purchase order
new books
book record
Book Supplier
D2 Books

DFD Level 0



DFD Level 1 (process 3.0 Issue and Return)



ER Diagram

Result:

The library information system was designed with the requirements, context diagram, DFD levels 0 and 1, ER diagram and tables.

Conclusion:

Leveled DFDs and the ER diagram give a complete logical design from which the Books, Members, Borrowing and Staff tables are derived.

Experiment No. 6

Create a Use Case Diagram of the Library Management System using a UML tool.

Aim:

To create a use case diagram for the Library Management System with the actors Student and Librarian and the use cases Login, Issue Book and Return Book, connected by associations.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A use case diagram shows the functions (use cases) that a system provides and the actors (users or external systems) that interact with it. The system boundary is drawn as a rectangle, actors as stick figures outside it and use cases as ovals inside it. An association is a line between an actor and a use case it takes part in. The relationship include (dashed arrow with the stereotype <<include>>) shows that a use case always uses the behaviour of another use case.

Procedure / Algorithm:

1. Open the UML tool and create a new Use Case diagram.
2. Draw the system boundary and name it Library Management System.
3. Add the actors Student and Librarian outside the boundary.
4. Add the use cases Login, Issue Book, Return Book (and Search Book, Pay Fine) inside the boundary.
5. Connect the actors with the use cases using associations and add <<include>> from Issue Book and Return Book to Login.

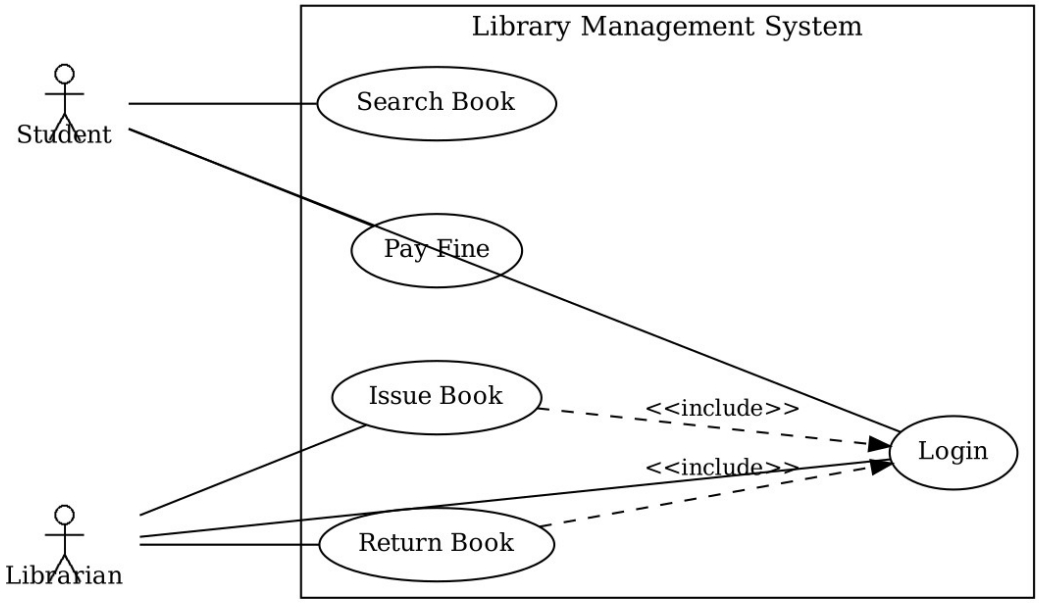
Actors, Use Cases and Associations:

Actor	Associated use cases	Description
Student	Login, Search Book, Pay Fine	Member who searches books, logs in and pays the fine.
Librarian	Login, Issue Book, Return Book	Staff member who issues and receives the books.

Use case	Description
Login	Authenticates the user with user name and password.
Issue Book	Librarian issues an available book to a member (includes Login).
Return Book	Librarian accepts the returned book and updates the fine (includes Login).
Search Book	Student searches the catalogue by title or author.
Pay Fine	Student pays the fine for a late return.

Output:

The use case diagram below shows the two actors, five use cases, the associations and the <<include>> relationships.



Use Case Diagram - Library Management System

Result:

The use case diagram of the Library Management System was created.

Conclusion:

A use case diagram gives a simple, user-oriented view of what the system does and who uses it.

Experiment No. 7

Draw a Class Diagram using a UML tool.

Aim:

To draw a class diagram with the classes Student, Book and Library, adding attributes and methods and showing association, aggregation, composition and inheritance.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A class diagram shows the static structure of a system: classes with their attributes and methods, and the relationships between them. A class is drawn as a box with three compartments (name, attributes, operations); + means public, - private and # protected. Association is a simple link with multiplicity; aggregation (hollow diamond) is a whole-part relationship in which the part can exist independently; composition (filled diamond) is a strong whole-part relationship in which the part cannot exist without the whole; inheritance (hollow triangle arrow) shows that a subclass is a specialisation of a superclass.

Procedure / Algorithm:

1. Create a new Class diagram in the UML tool.
2. Add the classes Person, Student, Librarian, Book, Library and Shelf.
3. Add the attributes and methods with their visibility and types.
4. Draw inheritance from Student and Librarian to Person.
5. Draw the association Student-Book, aggregation Library-Book and composition Library-Shelf and set the multiplicities.

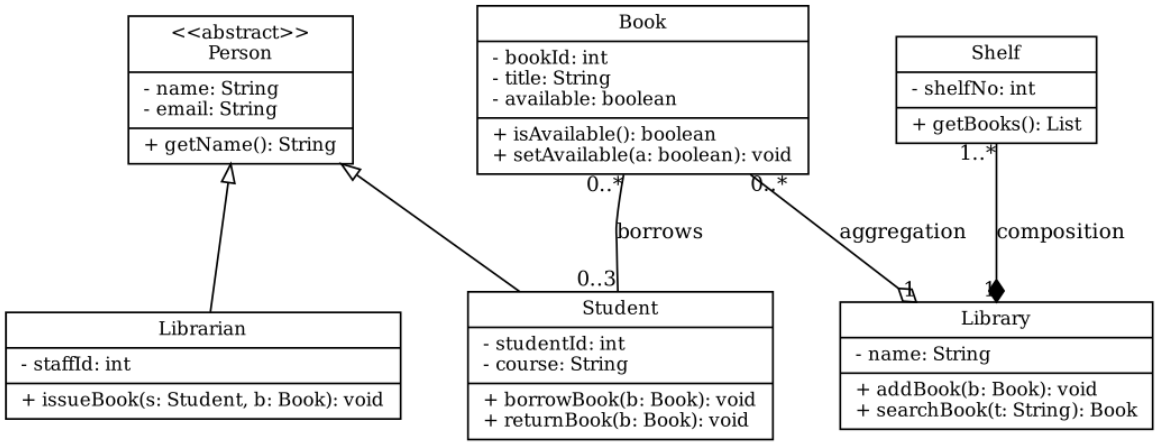
Classes and Relationships:

Class	Attributes	Methods
Person (abstract)	- name, - email	+ getName()
Student	- studentId, - course	+ borrowBook(b), + returnBook(b)
Librarian	- staffId	+ issueBook(s, b)
Book	- bookId, - title, - available	+ isAvailable(), + setAvailable(a)
Library	- name	+ addBook(b), + searchBook(t)
Shelf	- shelfNo	+ getBooks()

Relationship	Classes	Meaning
Inheritance	Student, Librarian -> Person	Student and Librarian are kinds of Person
Association	Student - Book (0..3 to 0..*)	A student borrows books
Aggregation	Library o-- Book (1 to 0..*)	Library has books; books can exist without the library
Composition	Library *-- Shelf (1 to 1..*)	Shelves exist only as part of the library

Output:

The class diagram below shows the six classes with attributes and methods and the four types of relationships.



Class Diagram - Library

Result:

The class diagram with association, aggregation, composition and inheritance was drawn.

Conclusion:

A class diagram models the structure of the system and the relationships between its classes before coding.

Experiment No. 8

Create a Sequence Diagram for the Book Issue process using a UML tool.

Aim:

To model the Book Issue process with a sequence diagram by adding objects, messages, activation bars and return messages.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A sequence diagram shows how objects interact in time order. Each object has a lifeline (vertical dashed line), time flows downwards, and messages are horizontal arrows between lifelines. A solid arrow with a filled head is a synchronous call, a dashed arrow is a return message, and a narrow rectangle on the lifeline is an activation bar which shows when the object is performing an action.

Procedure / Algorithm:

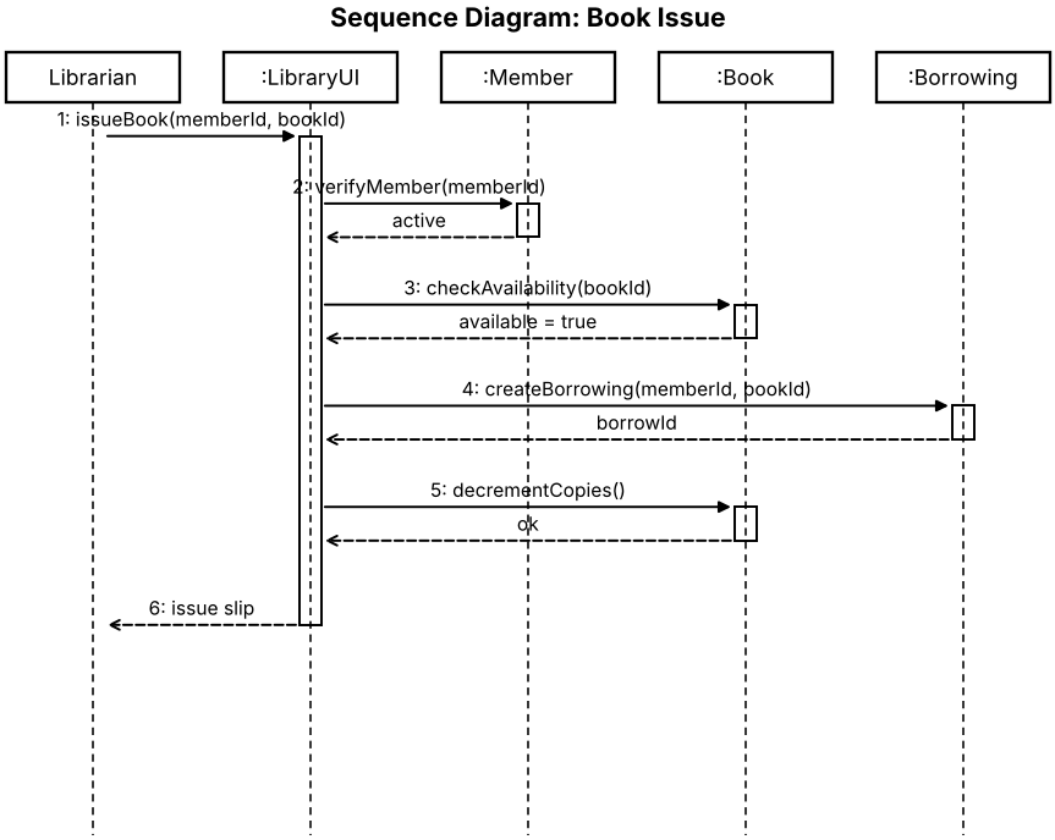
1. Create a new Sequence diagram.
2. Add the objects Librarian, :LibraryUI, :Member, :Book and :Borrowing.
3. Draw the messages in order from top to bottom: issueBook, verifyMember, checkAvailability, createBorrowing, decrementCopies.
4. Add the return messages as dashed arrows.
5. Add the activation bars on the lifelines and number the messages.

Objects and Messages:

No.	From	To	Message
1	Librarian	LibraryUI	issueBook(memberId, bookId)
2	LibraryUI	Member	verifyMember(memberId); returns active
3	LibraryUI	Book	checkAvailability(bookId); returns available = true
4	LibraryUI	Borrowing	createBorrowing(memberId, bookId); returns borrowId
5	LibraryUI	Book	decrementCopies(); returns ok
6	LibraryUI	Librarian	issue slip (return message)

Output:

The sequence diagram below shows the objects, the messages in execution order, the activation bars and the return messages of the Book Issue process.



Sequence Diagram - Book Issue

Result:

The sequence diagram of the Book Issue process was created.

Conclusion:

A sequence diagram clearly shows the order of messages between objects in a scenario.

Experiment No. 9

Create a Collaboration (Communication) Diagram using a UML tool.

Aim:

To represent the interactions between objects in the Book Issue scenario with a collaboration (communication) diagram and to number the messages in the order of execution.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A collaboration (communication) diagram shows the objects of an interaction as boxes, the links between them as lines and the messages as arrows along the links with sequence numbers (1, 2, 3 ...). It contains the same information as a sequence diagram, but it emphasises the structural organisation of the objects, whereas the sequence diagram emphasises the time order.

Procedure / Algorithm:

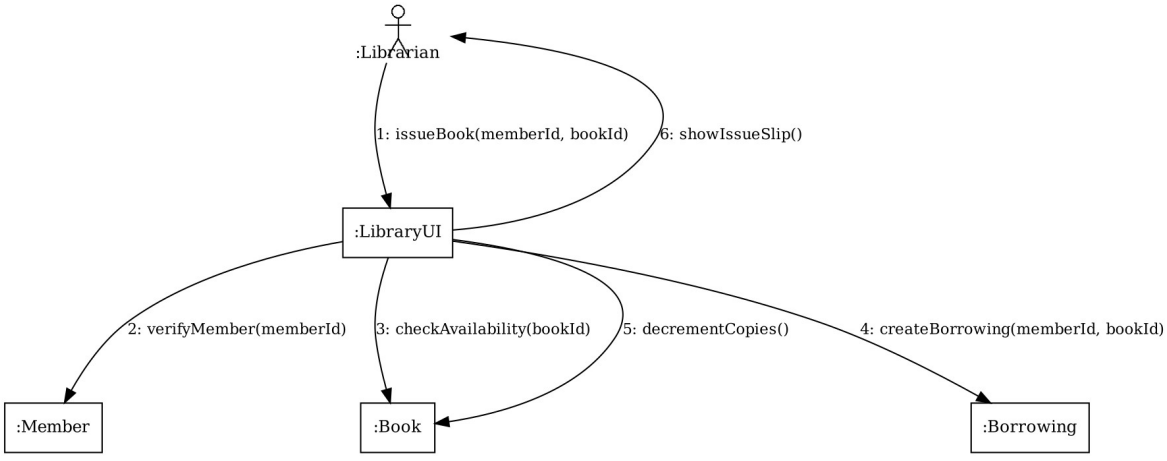
1. Create a new Collaboration (Communication) diagram.
2. Add the objects :Librarian, :LibraryUI, :Member, :Book and :Borrowing.
3. Draw the links between the objects that communicate.
4. Add the messages as arrows along the links.
5. Number the messages 1, 2, 3 ... in the order of execution.

Objects and Numbered Messages:

No.	From	To	Message
1	Librarian	LibraryUI	issueBook(memberId, bookId)
2	LibraryUI	Member	verifyMember(memberId)
3	LibraryUI	Book	checkAvailability(bookId)
4	LibraryUI	Borrowing	createBorrowing(memberId, bookId)
5	LibraryUI	Book	decrementCopies()
6	LibraryUI	Librarian	showIssueSlip()

Output:

The communication diagram below shows the objects, their links and the six numbered messages in execution order.



Collaboration (Communication) Diagram - Book Issue

Result:

The collaboration diagram with numbered messages was created.

Conclusion:

A communication diagram shows how objects are linked and which messages they exchange in an interaction.

Experiment No. 10

Draw an Activity Diagram for an ATM withdrawal process using a UML tool.

Aim:

To model the ATM cash withdrawal process using an activity diagram with an initial node, decision nodes, merge nodes and a final node.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

An activity diagram models the workflow of a process as a flow of activities. The initial node (filled circle) is the start, rounded rectangles are actions, arrows are control flows, a decision node (diamond) chooses one of several flows using guard conditions such as [yes] / [no], a merge node (diamond) joins alternative flows into one, and the final node (circle with a dot inside) ends the activity.

Procedure / Algorithm:

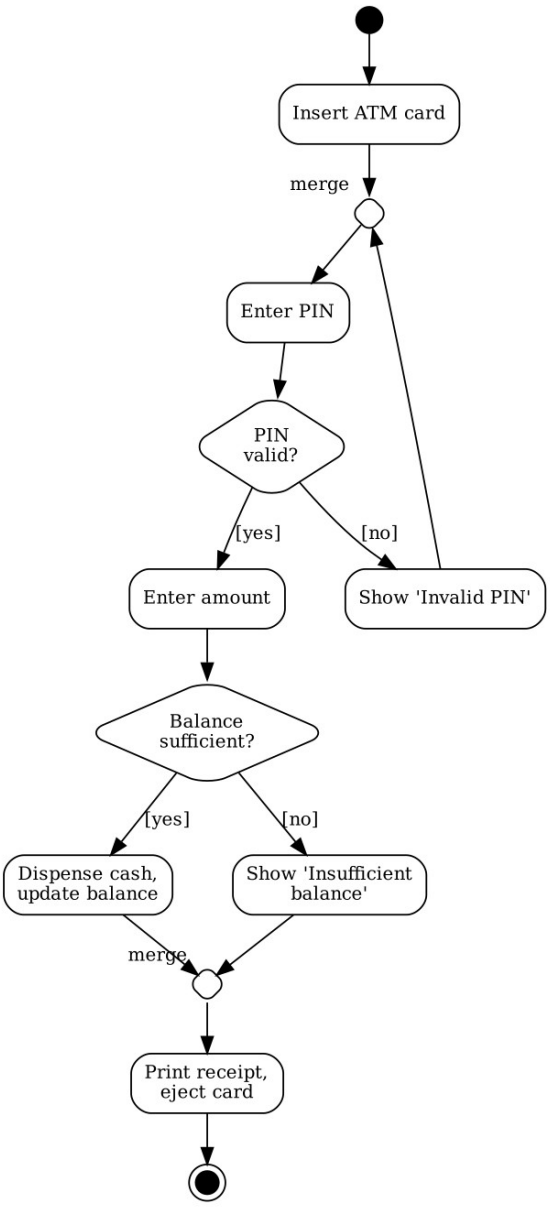
1. Create a new Activity diagram.
2. Add the initial node and the action Insert ATM card.
3. Add a merge node before Enter PIN to allow a retry, then add the decision node PIN valid?.
4. Add the actions and the second decision Balance sufficient? with the guards [yes] and [no].
5. Add a merge node to join the two paths, the action Print receipt, eject card and the final node.

Nodes of the Activity Diagram:

Node type	Name	Description
Initial node	Start	Process starts when the customer comes to the ATM
Action	Insert ATM card, Enter PIN, Enter amount	Steps done by the customer
Merge node	M1	Joins the first entry and the PIN retry flow
Decision node	PIN valid?	[yes] go to Enter amount, [no] show Invalid PIN and retry
Decision node	Balance sufficient?	[yes] dispense cash, [no] show Insufficient balance
Merge node	M2	Joins the dispense and the error paths
Action	Print receipt, eject card	Finishes the transaction
Final node	End	Process ends

Output:

The activity diagram below shows the ATM withdrawal flow with the initial node, two decision nodes, two merge nodes and the final node.



Activity Diagram - ATM Withdrawal

Result:

The activity diagram of the ATM withdrawal process was drawn.

Conclusion:

An activity diagram describes the flow of a process, including its decisions and alternative paths.

Experiment No. 11

Draw a State Chart Diagram for modeling the states of an ATM card using a UML tool.

Aim:

To draw a state chart diagram that models the states of an ATM card in the order of its life.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A state chart (state machine) diagram shows the states of one object during its life and the events that cause the object to change from one state to another. A state is drawn as a rounded rectangle, a transition as an arrow labelled with the event (and an optional action after /), the initial state as a filled circle and the final state as a circle with a dot inside. A transition from a state to itself is a self-transition.

Procedure / Algorithm:

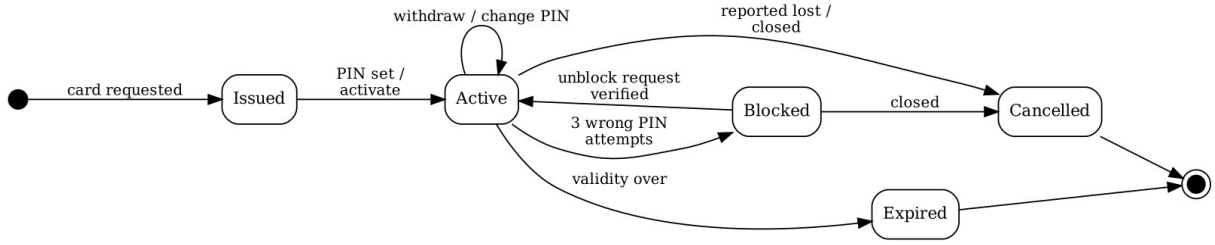
1. Create a new State Chart diagram.
2. Add the initial state and the states Issued, Active, Blocked, Expired and Cancelled.
3. Draw the transitions with their events, in the order of the card's life.
4. Add a self-transition on Active for normal use.
5. Add the final state reached from Expired and Cancelled.

States and Transitions:

From state	Event / action	To state	Order
Initial	card requested	Issued	1
Issued	PIN set / activate	Active	2
Active	withdraw / change PIN	Active	3
Active	3 wrong PIN attempts	Blocked	4
Blocked	unblock request verified	Active	5
Active	validity over	Expired	6
Active / Blocked	reported lost / closed	Cancelled	7
Expired / Cancelled	-	Final	8

Output:

The state chart diagram below shows the five states of an ATM card and the events that change the state.



State Chart Diagram - ATM Card

Result:

The state chart diagram of the ATM card was drawn.

Conclusion:

A state chart diagram clearly shows the life cycle of an object and the events that change its state.

Experiment No. 12

Create a Component Diagram using a UML tool.

Aim:

To create a component diagram that represents the software modules and the dependencies between the components of the library system.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A component diagram shows the physical modules of a software system (components) and the dependencies between them. A component is a replaceable part such as a library, package or executable that provides services through interfaces. It is drawn as a rectangle with the stereotype <<component>>, and a dashed arrow from component A to component B shows that A depends on B (A uses B).

Procedure / Algorithm:

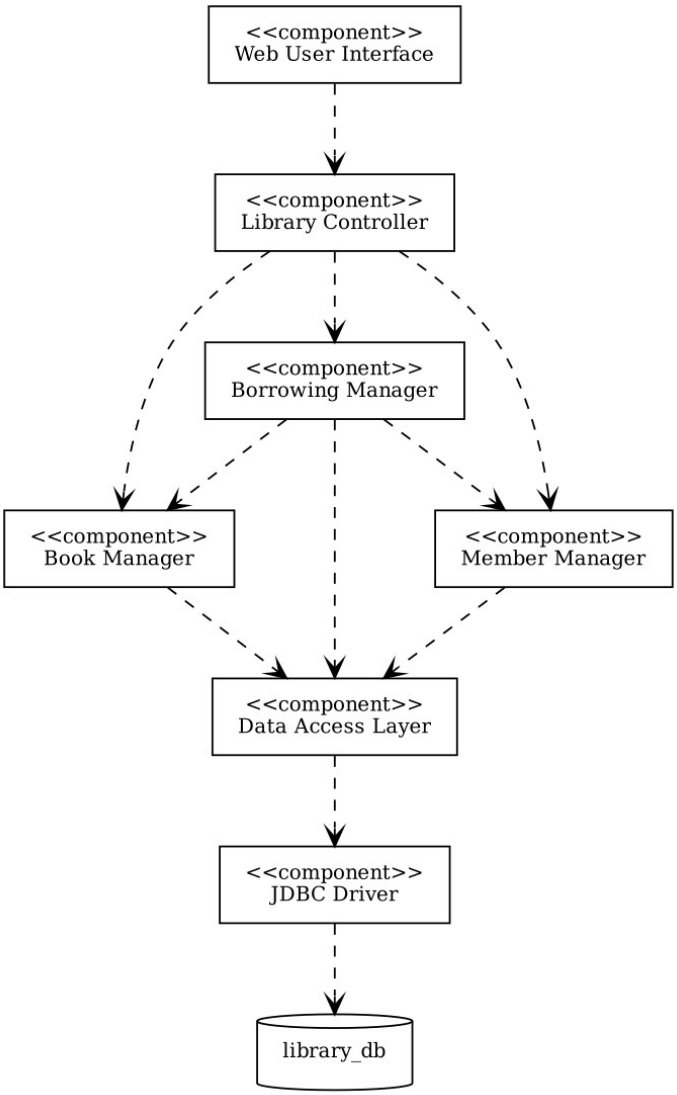
1. Create a new Component diagram.
2. Add the components Web User Interface, Library Controller, Book Manager, Member Manager, Borrowing Manager, Data Access Layer and JDBC Driver.
3. Add the database library_db as an external artifact.
4. Draw dependency arrows from each component to the components it uses.
5. Check that no component depends on a higher layer.

Components and Dependencies:

Component	Depends on
Web User Interface	Library Controller
Library Controller	Book Manager, Member Manager, Borrowing Manager
Borrowing Manager	Book Manager, Member Manager, Data Access Layer
Book Manager, Member Manager	Data Access Layer
Data Access Layer	JDBC Driver
JDBC Driver	library_db (MySQL database)

Output:

The component diagram below shows the seven software components, the database and the dependencies between them.



Component Diagram - Library System

Result:

The component diagram of the library software was created.

Conclusion:

A component diagram shows the modules of the software and their dependencies, which helps in planning the implementation and maintenance.

Experiment No. 13

Create a Deployment Diagram using a UML tool.

Aim:

To create a deployment diagram that shows the client computer, the application server and the database server and connects the nodes with communication links.

Software / Hardware Required:

UML tool (StarUML 5.x / Rational Rose / draw.io); PC with minimum 4 GB RAM, Windows / Linux.

Theory:

A deployment diagram shows the physical architecture of a system: the hardware nodes (drawn as 3-D boxes) and the software artifacts that run on them, together with the communication links between the nodes (lines labelled with the protocol). Typical nodes are client computers, application servers and database servers.

Procedure / Algorithm:

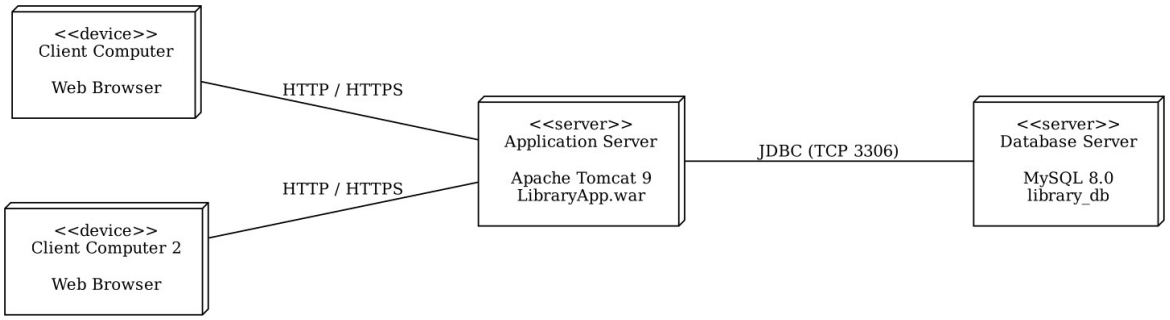
1. Create a new Deployment diagram.
2. Add the nodes Client Computer (with a web browser), Application Server (Apache Tomcat with LibraryApp.war) and Database Server (MySQL with library_db).
3. Draw communication links between the client and the application server (HTTP / HTTPS).
4. Draw the link between the application server and the database server (JDBC over TCP port 3306).
5. Add the artifacts and the stereotypes <<device>> and <<server>>.

Nodes and Communication Links:

Node	Artifacts / software	Link
Client Computer	Web browser	HTTP / HTTPS to the application server
Application Server	Apache Tomcat 9, LibraryApp.war	JDBC (TCP 3306) to the database server
Database Server	MySQL 8.0, library_db	-

Output:

The deployment diagram below shows two client computers, the application server and the database server with their communication links.



Deployment Diagram - Library System

Result:

The deployment diagram was created with the nodes and the communication links.

Conclusion:

A deployment diagram shows how the software is distributed on hardware nodes and how the nodes communicate.

Experiment No. 14

Import Java code into Rational Rose and generate the class diagram.

Aim:

To import (reverse engineer) Java source code into Rational Rose and to generate the class diagram from it.

Software / Hardware Required:

IBM Rational Rose (Enterprise Edition, with Java support), JDK 8 or later; PC with minimum 4 GB RAM, Windows.

Theory:

Reverse engineering is the process of creating a model from existing code. Rational Rose can read Java source files and create the classes, attributes, methods and relationships in the Logical View of the model, and the classes can then be dropped on a class diagram. The code must compile and the CLASSPATH must be set so that Rose can find the files. Reverse engineering keeps the model consistent with the code.

Procedure / Algorithm:

1. Write and compile the Java program (LibraryApp.java) given below.
2. Start Rational Rose and create a new model (File > New).
3. Select Tools > Java / J2EE > Project Specification and set the CLASSPATH to the folder of the source file.
4. Select Tools > Java / J2EE > Reverse Engineer, choose LibraryApp.java and click Reverse.
5. Open the Logical View, create a new class diagram and drag the classes Person, Student, Book and Library on it; the relationships appear automatically.

Java Source Code (LibraryApp.java):

```
import java.util.ArrayList;

class Person {
    private String name;
    private String email;
    public Person(String name, String email) { this.name = name; this.email = email; }
    public String getName() { return name; }
    public String getEmail() { return email; }
}

class Student extends Person {
    private String course;
    public Student(String name, String email, String course) {
        super(name, email);
        this.course = course;
    }
    public String getCourse() { return course; }
}

class Book {
    private int id;
    private String title;
    private boolean available = true;
    public Book(int id, String title) { this.id = id; this.title = title; }
    public String getTitle() { return title; }
    public boolean isAvailable() { return available; }
    public void setAvailable(boolean a) { available = a; }
    public int getId() { return id; }
}

class Library {
    private ArrayList<Book> books = new ArrayList<>();
    public void addBook(Book b) { books.add(b); }
    public boolean issueBook(int id) {
        for (Book b : books)
            if (b.getId() == id && b.isAvailable()) { b.setAvailable(false); return
true; }
```

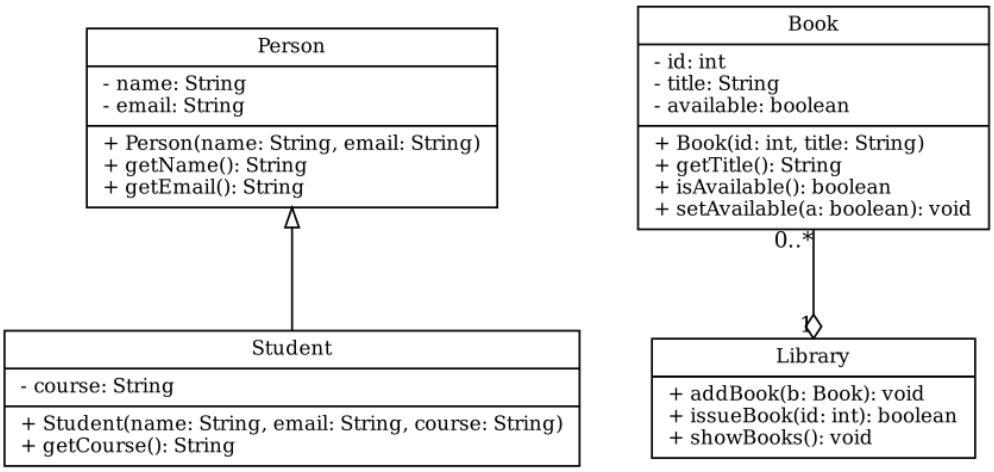
```
        return false;
    }
    public void showBooks() {
        for (Book b : books)
            System.out.println(b.getTitle() + " - " + (b.isAvailable() ? "Available" :
"Issued"));
    }
}

public class LibraryApp {
    public static void main(String[] args) {
        Student s = new Student("Rohit", "rohit@davv.ac.in", "M.Tech");
        Library lib = new Library();
        lib.addBook(new Book(1, "Database System Concepts"));
        lib.addBook(new Book(2, "Software Engineering"));
        System.out.println("Student: " + s.getName() + " (" + s.getCourse() + ")");
        System.out.println("Issue book 1: " + lib.issueBook(1));
        System.out.println("Issue book 1 again: " + lib.issueBook(1));
        lib.showBooks();
    }
}
```

Output:

```
Student: Rohit (M.Tech)
Issue book 1: true
Issue book 1 again: false
Database System Concepts - Issued
Software Engineering - Available
```

Console output of the program is given above. On reverse engineering, Rose created the classes Person, Student, Book and Library with their attributes and methods, the inheritance Student -> Person and the aggregation Library -> Book. The generated class diagram is shown below.



Class diagram generated from the Java code

Result:

The Java code was imported into Rational Rose and the class diagram was generated.

Conclusion:

Reverse engineering quickly converts existing code into a model, which helps in understanding and documenting a system.

Experiment No. 15

Create a class diagram and generate Java source code.

Aim:

To create a class diagram for an online shopping system in Rational Rose and to generate the Java source code from it.

Software / Hardware Required:

IBM Rational Rose (Enterprise Edition, with Java support), JDK 8 or later; PC with minimum 4 GB RAM, Windows.

Theory:

Forward engineering is the process of generating code from a model. After the classes, attributes, methods and relationships are drawn in a Rose class diagram, the option Generate Code creates a .java file for every class: the attributes become fields, the methods become empty method bodies, generalization becomes extends, an association or composition becomes a field of the related class type, and an abstract class or method is generated with the keyword abstract. The designer then writes the method bodies.

Procedure / Algorithm:

1. Create a new model in Rational Rose and open the Class Diagram in the Logical View.
2. Draw the classes Customer, Order, OrderItem, Product, Payment (abstract), CardPayment and CashPayment with attributes and methods.
3. Draw the associations, the composition Order-OrderItem and the inheritance of CardPayment and CashPayment from Payment.
4. Set the CLASSPATH in Tools > Java / J2EE > Project Specification.
5. Select all classes and choose Tools > Java / J2EE > Generate Code; check the generated .java files and compile them.

Generated Java Source Code:

```
import java.util.ArrayList;
import java.util.Date;

class Customer {
    private int customerId;
    private String name;
    public Order placeOrder() { return null; }
}

class Order {
    private int orderId;
    private Date orderDate;
    private ArrayList<OrderItem> items = new ArrayList<OrderItem>();
    private Customer customer;
    private Payment payment;
    public void addItem(Product p, int qty) { }
    public double getTotal() { return 0; }
}

class OrderItem {
    private int quantity;
    private Product product;
    public double getAmount() { return 0; }
}

class Product {
    private int productId;
    private String name;
    private double price;
    public double getPrice() { return 0; }
}
```

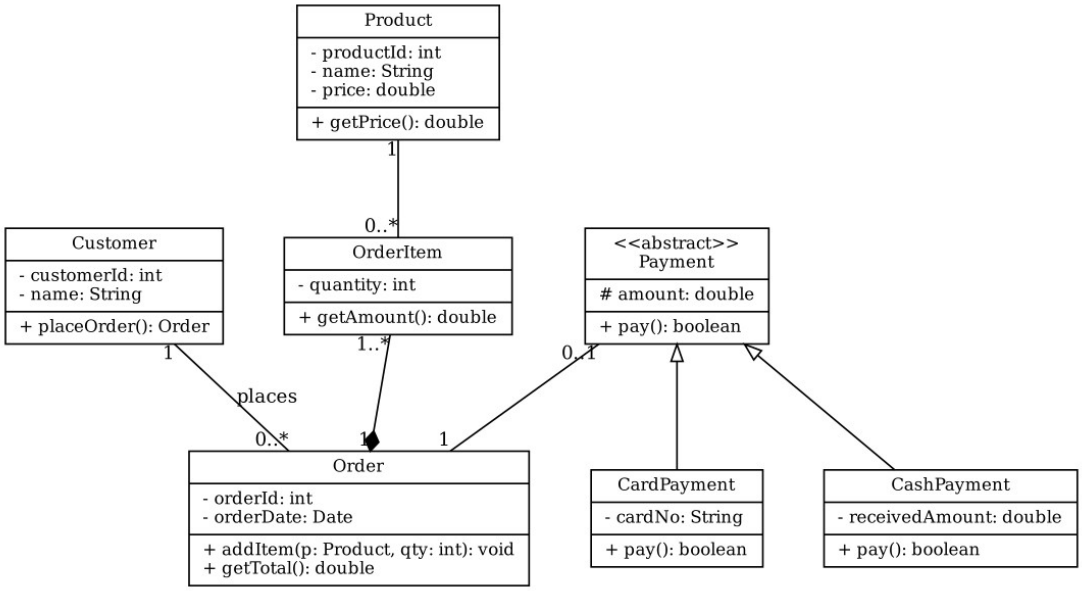
```
abstract class Payment {
    protected double amount;
    public abstract boolean pay();
}

class CardPayment extends Payment {
    private String cardNo;
    public boolean pay() { return false; }
}

class CashPayment extends Payment {
    private double receivedAmount;
    public boolean pay() { return false; }
}
```

Output:

The class diagram of the online shopping system is shown below. Rose generated seven .java files; the attributes became private fields, the composition and associations became fields of the related class (items, customer, payment) and the generalization became extends. The generated code compiled without errors.



Class Diagram - Online Shopping

Result:

The class diagram was created and the Java source code was generated from it.

Conclusion:

Forward engineering saves effort by generating the class structure from the design, and keeps the code and the model consistent.

Experiment No. 16

Create a new Rational Rose project, save the model and add a new diagram.

Aim:

To create a new Rational Rose project, to save the model and to add a new diagram to it.

Software / Hardware Required:

IBM Rational Rose (Enterprise Edition, with Java support), JDK 8 or later; PC with minimum 4 GB RAM, Windows.

Theory:

A Rational Rose model is stored in a single .mdl file and is organised in four views: the Use Case View (actors and use cases), the Logical View (classes and packages), the Component View (software components) and the Deployment View (hardware nodes). Every view can contain diagrams, which are added from the browser tree by right-clicking a package and choosing New and the diagram type. A model must be saved with File > Save.

Procedure / Algorithm:

1. Start Rational Rose; in the New Model dialog select a framework (or Cancel for an empty model) and click OK.
2. Examine the four views in the model browser on the left.
3. Select File > Save As, choose a folder, type the name LibraryModel and click Save (the file LibraryModel.mdl is created).
4. Right-click the Logical View, choose New > Class Diagram and name it Library Classes.
5. Open the new diagram, add a class from the toolbox, save the model again with File > Save.

Model Structure after the Steps:

View	Contents	Added in this experiment
Use Case View	Main diagram, actors, use cases	-
Logical View	Main class diagram, packages, classes	New class diagram "Library Classes" with one class
Component View	Main component diagram	-
Deployment View	Deployment diagram, nodes	-

Output:

The model was saved as LibraryModel.mdl and the new class diagram "Library Classes" appeared under the Logical View in the browser. After adding a class and saving again, the model opened correctly with the new diagram.

Result:

A new Rational Rose project was created, saved as a model and a new diagram was added.

Conclusion:

Rational Rose organises a project into views and diagrams in one model file, which can be saved and extended at any time.